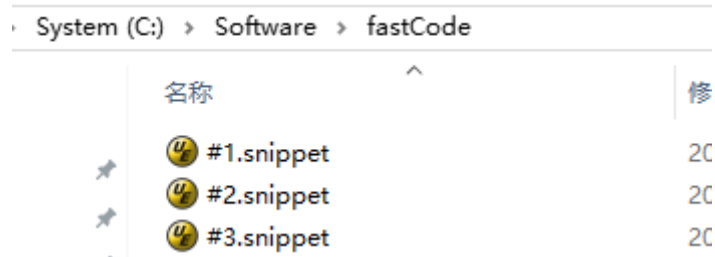


配置 VS2019 快捷导入代码

- 准备 快捷导入代码的脚本文件，保存在系统目录中（位置自定义）



- 在 VS2019 中配置，使用上述目录中的 脚本文件。
 - 工具 —— 代码片段管理器 —— 修改 Basic 为 Visual C++ —— 选择 上述自定义的目录位置（不需要选择到具体脚本文件）。
- 在程序中使用 快捷导入代码。
 - #1 ---- tab 键。

for 循环

语法

```
1 for (表达式1;表达式2;表达式3)
2 {
3     循环体。
4 }
```

- 循环从 表达式1开始 ——> 表达式2 (判别表达式) ——> 真 ——> 执行循环体 ——> 表达式3 ——> 判断表达式2
 - 真：继续 ——> 循环体 ——> 表达式3 ——> 表达式2。。。
 - 假：跳出循环。（正常情况下，for循环的出口是 表达式2）

练习：

- 使用 for 循环 求 1~100 的和。

```
1 // 1~100 求和          1+2+3+4+5+。。。+100
2 int main(void)
3 {
4     // 定义循环因子。
5     int i = 0;          // 定义i，给初值。
6
7     // 定义变量 记录累加值
8     int sum = 0;        // 初值为 0
9     for (i = 1; i <= 100; i++)
10    {
11        sum = sum + i;
12    }
```

```

13 // 循环结束，打印 累加结果
14 printf("sum = %d\n", sum);
15
16 return EXIT_SUCCESS;
17 }

```

for 循环的变换形式

- 循环因子 i:
 - 在 for 循环之前定义。在 for 循环，结束后依然能使用。
 - 定义在 for 循环之内。for 循环结束后，不能使用！

```

1 for (int i = 1; i <= 100; i++) // 将 i 的定义放到 for 内 表达式1上。
2 {
3     sum = sum + i;
4 }
5
6 // 循环结束，打印 累加结果
7 printf("sum = %d, i = %d\n", sum, i); // 编译器保存，“未定义标识符”

```

- for 循环 3 个表达式，在使用时，均可省略。但，2 个“;”不允许省略。

1. 省略 表达式1

```

1 int i = 1; // 定义 循环因子
2 int sum = 0;
3
4 for (; i <= 100; i++) // 不写表达式1
5 {
6     sum = sum + i;
7 }

```

2. 省略 表达式3

```

1 int i = 1; // 定义 循环因子
2 int sum = 0;
3
4 for ( ; i <= 100; ) // 不写表达式1，不写表达式3
5 {
6     sum = sum + i;
7     i++; // 将原来的表达式3写到循环体中。
8 }

```

3. 省略 表达式2

```

1  int i = 1;  // 定义 循环因子
2
3  for ( ; ; ) // 不写表达式2, 相当于 for ( ; 1 ; )表达式2为 真(1)。 这是一个
    死循环。
4  {
5      printf("i = %d\n", i);
6      i++;
7  }
8  // for ( ; ; ) 死循环 相当于 while(1) {};

```

- for 每个表达式中，可以含有多个算式。

```

1  int i = 0;  // 定义 循环因子
2  int a = 0;
3
4  for (i=1, a=3; a<20, i<10; i++, a+=3) // i<10, a <20 也可以写成 i<10 &&
    a<20
5  {
6      printf("i = %d\n", i);
7      printf("a = %d\n", a);
8  }

```

- 死循环

```

1  // 方法1:
2  for(;;)
3
4  // 方法2:
5  while(1)

```

练习:

- 猜数字游戏:
 - 产生一个随机数。用户键盘键入一个数据，程序提示用户，输入的数据 > < == 随机数。用户根据提示不断变换输入，最终猜中!
- 生成随机数:
 1. 添加一个随机数种子。作用：保证随机是真正的随机。

```

1  srand(time(NULL));  // 固定写法。
2
3  // time(NULL): 获取系统当前时间。unsigned long long 类型。
4  // srand() 函数来生成随机数。使用 系统时间为 算法的系数。

```

2. 添加头文件

```

1  // srand() --- <stdlib.h>
2  // time() --- <time.h>

```

3. 生成随机数。

```

1  int n = rand() % 100;           // 随机数范围: 0 ~ 99
2  int n = rand() % 100 + 13;      // 随机数范围: 13 ~ 112
3  int n = rand() % 63 + 17;       // 随机数范围: 0+17 ~ 62+17

```

• 思路分析

```

1  // 伪代码
2  int main(void)
3  {
4      1. 生成随机数。(3步骤)
5      2. 创建死循环, 供用户猜测    while(1)
6          int num ;
7          while (1)
8          {
9              接收用户输入: scanf("%d", &num);
10             if 判断用户输入的数据, 与实际随机 大小。
11             num > 随机数
12                 提示用户。继续循环。
13             num < 随机数
14                 提示用户。继续循环。
15             num == 随机数
16                 提示用户 猜中! 结束循环。
17                 break; 跳出循环。
18         }
19     return 0;
20 }

```

• 编码实现

```

1  int main(void)
2  {
3      // 播种随机数种子
4      srand(time(NULL));
5
6      // 生成随机数 n
7      int n = rand() % 100; // 范围 0-99
8
9      // 定义 num 变量, 存储用户输入的数据。
10     int num;
11
12     // 创建 死循环, 给用户猜数字。
13     for (;;) // while(1) 等价
14     {
15         printf("请输入猜测的数字: ");
16         // 获取用户输入数据
17         scanf("%d", &num);
18
19         // 提示用户, 测试方向
20         if (num > n)
21         {
22             printf("猜大了! \n");
23         }
24         // 如果 if分支、for、while 满足后, 执行语句只有一条时, {} 可以省略。
25         else if (num < n)
26
27             printf("猜小了! \n"); // 语法允许, 不写 {}

```

```

28
29         else
30         {
31             printf("猜中了!!! \n");
32             break; // 不必再循环。
33         }
34     }
35     printf("本尊数: %d\n", n);
36
37     return EXIT_SUCCESS;
38 }

```

嵌套 for 循环

```

1  int i = 0; // 外层循环的循环因子
2  int j = 0; // 内层循环的循环因子
3
4  for (i = 0; i < 10; i++)
5  {
6      for (j = 2; j < 10; j++)
7      {
8          // 循环体
9      }
10 }

```

- 结论：外层循环执行一次，内层循环执行一周。

练习：

- 模拟电子表打印
- 分析：

```

1  // 最外层
2  for (i = 0; i < 24; i++)
3  {
4      for (j = 0; j < 60; j++)
5      {
6          for (k = 0; k < 60; k++)
7          {
8              打印时间。
9          }
10     }
11 }
12 11:30:54
13 11:30:56
14 ...
15 11:31:00
16 ...
17 12:00:00
18 12:00:01

```

- 实现:

```
1  int main(void)
2  {
3      int i, j, k;
4
5      for (i = 0; i < 24; i++) // 小时
6      {
7          for (j = 0; j < 60; j++) // 分钟
8          {
9              for (k = 0; k < 60; k++) // 秒
10             {
11                 printf("%02d:%02d:%02d\n", i, j, k);
12                 sleep(980);
13                 system("cls"); // 清屏
14             }
15         }
16     }
17
18     return EXIT_SUCCESS;
19 }
```

练习:

- 打印正序 9x9 乘法表

```
1  1x1= 1           // 第1行, 打印1列
2  1x2= 2 2x2= 4     // 第2行, 打印2列
3  1x3= 3 2x3= 6 3x3= 9 // 第3行, 打印3列
4  1x4= 4 2x4= 8 3x4=12 4x4=16 // 第4行, 打印4列
5  ...
6  1x9= 9 2x9=18 3x9=27 4x9=36 5x9=45 6x9=54 7x9=63 8x9=72 9x9=81 // 第9
   行, 9列
7
8  jxi =
9  第 i 行, 打印 i 列。
```

- 分析+实现

```
1  for (i = 1; i <= 9; i++) // 外层, 描述行。
2  {
3      for (j = 1; j <= i; j++) // 内层, 描述每一列。
4      {
5          printf("%dx%d=%d\t", j, i, i*j);
6      }
7      printf("\n");
8  }
```

- 打印倒序 9x9 乘法表

```
1  1x9= 9 2x9=18 3x9=27 4x9=36 5x9=45 6x9=54 7x9=63 8x9=72 9x9=81 // 行
2  1x8= 8 2x8=16 3x8=24 4x8=32 5x8=40 6x8=48 7x8=56 8x8=64
3  1x7= 7 2x7=14 3x7=21 4x7=28 5x7=35 6x7=42 7x7=49
```

```

4  1x6= 6 2x6=12 3x6=18 4x6=24 5x6=30 6x6=36
5  1x5= 5 2x5=10 3x5=15 4x5=20 5x5=25
6  1x4= 4 2x4= 8 3x4=12 4x4=16
7  1x3= 3 2x3= 6 3x3= 9
8  1x2= 2 2x2= 4
9  1x1= 1
10
11 jxi = 值
12
13 for (i = 9; i >= 1; i--)    // i 控制 行
14 {
15     for (j = 1; j<=i; j++)
16         printf("dx%d=%d\t", j, i, i*j);
17     putchar('\n');
18 }
19 // 上述省略{}写法无误， 不推荐。--- 推荐下面的写法。
20 int main(void)
21 {
22     int i, j;
23     for (i = 9; i >= 1; i--)
24     {
25         for (j = 1; j <= i; j++)
26         {
27             printf("dx%d=%d\t", j, i, j * i);
28         }
29         putchar('\n'); // printf("\n");
30     }
31     return EXIT_SUCCESS;
32 }

```

跳转语句

break

- 作用1:
 - 一次break, 可以跳出一重循环。(for、while、do while)
- 作用2:
 - 防止 case 穿透。 结束 switch();

continue

- 作用: 结束 **本次** 循环。continue关键字之后的代码, 在这次循环中, 不执行。
- 示例1:

The screenshot shows a C++ IDE with a code editor on the left and a console window on the right. The code in the editor is as follows:

```
{
    for (int i = 0; i < 5; i++)
    {
        printf("=====1=====\n");
        printf("=====2=====\n");
        if (i == 2) {
            continue;
        }
        printf("=====3=====\n");
        printf("=====4=====\n");
        printf("=====5=====\n");
        printf("\n");
    }

    return EXIT_SUCCESS;
}
```

The console window on the right shows the output of the program, which is a series of lines with 10 equals signs and a newline character, repeated five times. The output is as follows:

```
=====1=====
=====2=====
=====3=====
=====4=====
=====5=====
=====1=====
=====2=====
=====3=====
=====4=====
=====5=====
=====1=====
=====2=====
=====3=====
=====4=====
=====5=====
```

- 示例2:

```
1  int main(void)
2  {
3      for (int i = 0; i < 10; i++)
4      {
5          for (int j = 0; j < 5; j++)
6          {
7              if (j == 2)
8              {
9                  continue; // 只跳出（后续代码不执行）本次 j == 3 时的循环，
10             }
11             printf("i = %d, j = %d\n", i, j);
12         }
13         printf("\n");
14     }
15     return EXIT_SUCCESS;
16 }
17 }
```

goto

- 语法:

1. 设定一个标签。标签名自定义，一般大写。如：ABC、LABEL、AAA
2. 使用“goto 标签名”跳转到标签的位置。（只函数内生效）

- 示例:

```
1  int main(void)
2  {
3      printf("=====1=====\n");
4      printf("=====2=====\n");
```



```

5     printf("=====3=====\\n");
6
7     goto LABEL;
8     printf("=====4=====\\n");
9     printf("=====5=====\\n");
10    printf("=====6=====\\n");
11    printf("=====7=====\\n");
12
13 LABEL:
14    printf("=====8=====\\n");
15    printf("=====9=====\\n");
16
17    return EXIT_SUCCESS;
18 }

```

- 示例:

```

int main(void)
{
    int i = 0, j = 0;

    for (i = 0; i < 5; i++)
    {
        if (i == 2)
        {
            goto ABC234;
        }
        printf("i = %d\\n", i);
    }

    for(j = 0; j < 5; j++)
    {
        ABC234:
        printf("j = %d\\n", j);
    }

    return EXIT_SUCCESS;
}

```

选择Microsoft Visual Studio 调试控制台

```

i = 0
i = 1

```

```

j = 0
j = 1
j = 2
j = 3
j = 4

```

C:\itcast\VSsource\day05\Debug\day05.exe (进程 154...)
按任意键关闭此窗口...

j = 0 的表达式，没有执行。

- goto 语法过于灵活，会打乱程序的执行逻辑，降低代码的可读性。后续编程中，尽量少用。
 - C 程序中，简单的逻辑，依然可以使用 goto。

数组

- 什么是数组:
 - 数组是，相同数据类型有序的、连续的存储集合。

```
// 定义数组，定义10个类型相同、连续、有序存储的数据。
int arr[10] = {19, 2, 23, 4, 5, 6, 10, 7, 8, 9};

//printf("arr[0] = %d\n", arr[0]); // 取数组的第一个元素。

printf("&arr[0] = %p\n", &arr[0]); // 取数组的第一个元素的内存地址。
//printf("&arr[0] = %x\n", &arr[0]); // 内存地址使用 16 进制数表示。
//printf("&arr[0] = %#x\n", &arr[0]);
printf("&arr[1] = %p\n", &arr[1]);
printf("&arr[2] = %p\n", &arr[2]);
printf("&arr[3] = %p\n", &arr[3]);
```

选择Microsoft Visual Studio 调试控制台

```
&arr[0] = 008FF9C4
&arr[1] = 008FF9C8
&arr[2] = 008FF9CC
&arr[3] = 008FF9D0
```

- %p:
 - 用来打印变量内存地址的专用 格式匹配符 (占位符)

基本特性

1. 各个元素，连续存储。
2. 数组名为地址，是数组首个元素的地址。arr == &arr[0]
3. 求数组的总大小：

```
1 | printf("数组的大小: %u\n", sizeof(arr));
```

4. 求数组每一个元素的大小：

```
1 | printf("数组元素的大小: %u\n", sizeof(arr[0]));
```

5. 求数组元素的个数：

```
1 | printf("数组元素的个数: %d\n", sizeof(arr)/sizeof(arr[0]));
```

6. 数组第一个元素的下标：0
7. 数组最后一个元素的下标：

```
1 | sizeof(arr)/sizeof(arr[0]) - 1
```

数组初始化

```
1 // 初始化方法1
2 int arr[5] = {3, 7, 2, 1, 9};
3
4 // 初始化方法2【多用】
5 int arr[5] = {3, 7}; // 剩余未初始化的元素，默认值 0
6
7 // 初始化方法3【多用】
8 int arr[5] = {0}; // 初始化一个 全部元素为 0 的数据。 ---清0 常用。
9
10 // 初始化方法4【多用】
11 int arr[] = {3, 7, 2, 1, 6, 9, 13}; // 编译器会自动求取数组元素个数。
```

```

12
13 // 初始化方法5
14 int arr[] = {0}; // 定义了只有一个元素的数组，值为 0
15
16 // 初始化方法6【多用】
17 int arr[10]; // 声明了一个有10个元素数组。
18     arr[0] = 5;
19     arr[1] = 6;
20     arr[2] = 7; // 剩余未初始化的元素，默认值 —— 随机数。

```

练习

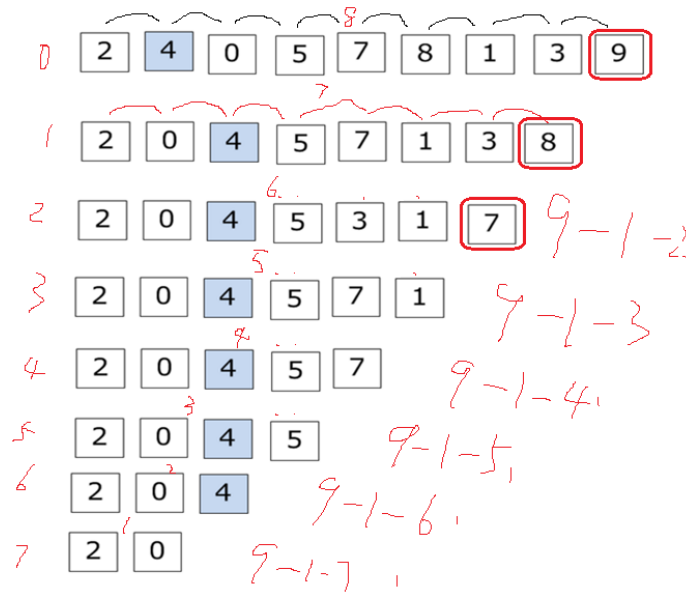
- 数组元素逆序

```

1 // 数组元素逆序
2 int main(void)
3 {
4     int arr[] = {1, 6, 8, 0, 4, 3, 9, 2}; // 变为: {2, 9, 3, 4, 0, 8, 6, 1}
5
6     // 获取数组的元素个数
7     int n = sizeof(arr) / sizeof(arr[0]);
8
9     int i = 0; // 从前向后
10    int j = n-1; // 从后向前
11    int tmp = 0; // 定义临时变量。
12
13    // 交换数组元素之前，打印数组的所有元素。
14    for (int i = 0; i < n; i++)
15    {
16        printf("%d ", arr[i]);
17    }
18    putchar('\n');
19
20    // 循环交换数组元素。
21    while (i < j)
22    {
23        tmp = arr[i]; // 三杯水变量交换法
24        arr[i] = arr[j];
25        arr[j] = tmp;
26        i++; // 不断后移
27        j--; // 不断前移
28    }
29
30    // 交换数组元素之后，打印数组的所有元素。
31    for (int i = 0; i < n; i++)
32    {
33        printf("%d ", arr[i]);
34    }
35    putchar('\n');
36
37    return 0;
38 }

```

冒泡排序



N 个数，比 N-1 行

```
for (i = 0; i < 9-1; i++) // 外层控制行
{
    for (j = 0; j < 9-1-i; j++)
    {
        if (arr[j] > arr[j+1])
        {
            tmp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = tmp;
        }
    }
}
```

```
1  int main(void)
2  {
3      int i, j, tmp;
4
5      int xjp[] = {12, 2, 32, 14, 62, 54, 27, 89, 9, 10, 3};
6
7      // 求取数组元素个数。
8      int n = sizeof(xjp) / sizeof(xjp[0]);
9
10     // 开始排序。
11     for (i = 0; i < n - 1; i++) // 外层控制行
12     {
13         for (j = 0; j < n - 1 - i; j++) // 内层控制列。
14         {
15             // 相邻两两比较，三杯水交换
16             if (xjp[j] > xjp[j + 1])
17             {
18                 tmp = xjp[j];
19                 xjp[j] = xjp[j + 1];
20                 xjp[j + 1] = tmp;
21             }
22         }
23     }
24     // 打印排序结果。
25     for (i = 0; i < n; i++)
26     {
27         printf("%d ", xjp[i]);
28     }
29     printf("\n");
30
31     return EXIT_SUCCESS;
32 }
```